

When a client instantiates a `SimpleAnimation` object, it must pass in the following:

window The window to draw into.

loc The location in the window to draw the images.

picPaths A list or tuple of paths to images. The images will be displayed in the order given here.

durationPerImage How long (in seconds) to show each image.

In the `__init__()` method ❶, we save these parameter variables into similarly named instance variables. The method loops through the list of paths, loads each image, and saves the resulting images into a list. A list is a perfect way to represent an ordered set of images. The class will use the `self.index` variable to keep track of the current image in the list.

The format of an image in a file is different from the format of an image when displayed on the screen. The call to `convert_alpha()` ❷ converts from the file format to the screen format to optimize performance when showing an image in the window. The actual drawing is done later, in the `draw()` method.

The `play()` method ❸ starts an animation running. It first checks to see if the animation is already running, and if it is, the method just returns. Otherwise, it sets `self.playing` to `True` to indicate that the animation is now running.

When a `SimpleAnimation` is created, the caller specifies the amount of time that each image should be shown, and this is saved in `self.durationPerImage`. Therefore, we must keep track of time as a `SimpleAnimation` runs to know when to switch to the next image. We call `time.time()` to get the current time (in milliseconds) and save that in an instance variable. Making the class time based means that any `SimpleAnimation` object built from this class will work correctly, independent of the frame rate used for the main loop. Finally, we set the variable `self.index` to 0 to indicate that we should be showing the first image.

The `update()` method ❹ needs to be called in every frame of the main loop. If the animation is not playing, `update()` does nothing and just returns.